



Multi-cloud security model: establishment of inter-server communication for authentication integrity

Sumeet Kaur¹ · Mayank Deep Khare² · Mukul Bhatt³ · V. Haripriya⁴ · Ajay Kumar⁵ · Abhishek Singla⁶

Received: 24 April 2024 / Revised: 26 August 2024 / Accepted: 27 September 2024

© The Author(s) under exclusive licence to The Society for Reliability Engineering, Quality and Operations Management (SREQOM), India and The Division of Operation and Maintenance, Lulea University of Technology, Sweden 2024

Abstract Safety measures are used in multi-cloud security to protect infrastructure, apps, and data that are spread across several cloud environments. It necessitates lowering the risks connected to dispersed data access and storage by maintaining honesty and secrecy across several cloud platforms. The purpose of this research is to develop a novel inter-server communication for securing a multi-cloud environment. We discuss several challenges associated with utilizing many clouds and offering solutions such as the Supersingular isogeny Diffie–Hellman (SIDH) algorithm to prevent sensitive messages from being intercepted during internal server communications, Attribute-based encryption with outsourced decryption (ABE-OD) for fine-grained access control, rekeying at user withdrawal, Blowfish for message encryption, Rivest, Shamir, and Adleman (RSA) for key encryption. Our suggested paradigm includes an exchange of keys methodology to strengthen server-to-server

prerequisites for communications in addition to security for fine-grained data security and access control on systems for storing. The RSA technique is used in the suggested concept to encrypt secret keys, allowing the encrypted keys to be stored in the database proxy server. The computation times for the construction of the BLAKE2 hash, data encryption, data decryption, and SIDH key exchange were determined. We made use of many parameters, including hash generation time, encryption time, and decryption time. The outcomes of the parameters are hash generation time of 2 mb (45), 4 mb (167), 8 mb (301), 16 mb (421), 22 mb (563), 28 mb (685), 32 mb (705), 36 mb (911), 42 mb (1027), encryption times of 2 mb (24), 4 mb (90), 8 mb (170), 16 mb (265), 22 mb (342), 28 mb (420), 32 mb (508), 36 mb (595), 42 mb (712), and decryption time of 2 mb (42), 4 mb (164), 8 mb (298), 16 mb (417), 22 mb (560), 28 mb (682), 32 mb (705), 36 mb (907), 42 mb (1024). Based on the inter-server communication system, the experiment results demonstrate the effectiveness of the recommended methodology in securing a multi-cloud environment.

✉ Sumeet Kaur
sumeetkaur012345@outlook.com;
sumeet.kaur.orp@chitkara.edu.in

¹ Centre of Research Impact and Outcome, Chitkara University, Rajpura, Punjab 140417, India

² Department of Computer Science and Engineering (IOT), Noida Institute of Engineering and Technology, Greater Noida, Uttar Pradesh, India

³ Department of ISME, ATLAS SkillTech University, Mumbai, Maharashtra, India

⁴ Department of Computer Science and Information Technology, Jain (Deemed to Be University), Bangalore, Karnataka, India

⁵ Department of Computer Science and Engineering, Vivekananda Global University, Jaipur, India

⁶ Chitkara Centre for Research and Development, Chitkara University, Baddi, Himachal Pradesh 174103, India

Keywords Multi-cloud security · Inter-server communication · Supersingular isogeny Diffie–Hellman (SIDH) · Data security

1 Introduction

This approach of computing is provided with the current cloud computing, which allows customers online access to platforms, software, and infrastructure. The premier security protection that may be provided by cloud systems should be protection against infrastructure, applications, and data compromise due to cyber-attacks (Yang et al. 2020). It applies the approach of cloud security, including encryption, access

control, and monitoring, to ensure the realization of cloud-based service privacy, mobility, and service integrity. It is particularly interested in contexts that use multiple clouds and in identity and access management. This is through the passing of instructions and data to communicate over the network, for example, servers between each other. This interface is key to the business processes and sharing of resources amid the cloud environments and all these multiple locations services that span these (Rajkumar et al. 2022a, b). Its main technology, which is utilized in such assurance, includes encryption and authentication of communication between the servers on the network using the Transport Layer Security (TLS)/Secure Sockets Layer (SSL) protocols and two-way authenticating functions.

Also, they are sure to have security traits that bar them from acquiring unwanted entry, tampering with information, and communication ruptures (Shufutinsky 2020). Secured through a wide variety of mechanisms, among them the issuance of online certificates for user protection (AlRousan and Intrigila 2020). It is a certificate in electronic form used to authorize a person to carry out transactions online. Biometric refers to an identity access technique that involves the use of unique behavioral evidence, for example, fingerprints, handprints, retina patterns, and even voiceprints. Just like security, multi-factored authentication is applied in a way that several combinations of authentication types are used, like codes for cell phones and passwords. This is thus supposed to be valid for locking access to prevent unauthorized access, prevention against identity theft, and protection from other security risks.

The Multi-Cloud Security Model brings inter-server verbal exchange, authentication integrity, cloud security, and multi-cloud safety collectively. It develops a defensive framework change of messages among servers in several cloud environments, as well as ensuring that authentication remains safe throughout implementation (Narayanan et al. 2022). Encryption key management is essential for security. To save intrusion or compromise, authentication solutions must make certain that keys are generated, saved, and appropriately disbursed (Harold Robinson and Golden Julie (2019)). Regular conversation between servers monitoring enables quick reaction and reduction in depth by way of assisting within the detection of abnormalities and feasible hazards to safety. Furthermore, restricting admission to authorized companies and blocking off unauthorized transactions, increases security (Kang and Lemieux (2021)). To make certain compliance with stated requirements, server communication has managed the usages of access management mechanisms. Identification and mitigation of intrusion systems improve protection by actively gazing and snooping on malicious behavior or unapproved entry. By stressing the importance of maintaining protecting towards information breaches and retaining authentication integrity and

unauthorized get admission, the studies seek to provide a thorough foundation for stable server trade in multi-cloud ecosystems. The primary goal of this investigation is to create a safe architecture for inter-server communication within multi-cloud systems. The purpose is to improve data security and accessibility control and maintaining effective key transmission.

1.1 Key contributions of this research

- The efficiency of the suggested architecture combines Public Key Infrastructure (PKI) with traditional cryptographic approaches such as Supersingular Isogeny Diffie–Hellman (SIDH), Blowfish, Rivest, Shamir, and Adleman (RSA), and BLAKE2.
- The proposed SIDH algorithm is used to secure inter-server communication, ensuring that sensitive messages exchanged between servers remain confidential and integral, preventing interception or tampering.
- This technique effectively addresses data security as well as user privacy concerns, demonstrating its appropriateness for real-time applications.
- The article presents a unique strategy for improving effectiveness, scalability, and robustness in multi-cloud environments.

1.2 Problem statement

Existing approaches, such as Attribute-Based Encryption (ABE) and standard cryptographic systems have concerns with message interception and scalability. This paper provides a strong security architecture which improves data protection, access management, and key transferring in dynamic cloud environments. The issue involves developing a system which efficiently mitigates these issues while responding to changing cloud infrastructures.

The following portions of this work are organized as follows. Section 2 includes related studies, while Sect. 3 covers essentials such as grouping, difficulty assumptions, and access structure. It also provides background on the ABE-OD algorithm, RSA, ECC, and SIDH, as well as the suggested system model. Section 4 includes performance outcomes. The paper comes to its conclusion in Sect. 5.

2 Related works

Vijay and Reddy (2021) presented a multi-cloud system that includes several strategies for internal server interaction security and data security that can effectively protect data privacy from monitors. The use of efficient secure algorithms, a modified Diffie–Hellman key-sharing method, and a dispersion approach ensured a system's overall

productivity with superior safety. The outcomes of the multi-cloud setup suggested the proposed framework's efficiency relative to the time spent on storing data, calculation, and retrieval while removing the risk of assaults. Mhaisen and Malluhi (2020) described consistent data in multi-cloud systems of storage, said that it could be broken, and proposed a new method for proving data consistency in such systems. The testing and implementation demonstrated that the suggested solution was scalable as a result of both the number of users as well as the number of clouds used and it kept data consistent with certain levels of latency in data uploads. Applying this technology to multi-cloud data centers would improve their usefulness and dependability.

Rajkumar et al. (2022a, b) proposed an innovative DL-based approach to logical network protocol design and performance evaluation. There is a cross-domain identity management system acting as a user authorization protocol for cloud computing users and providers. He said that the facilities required cloud computing to have a secure infrastructure at the customer's premises. This included data security at all levels, which would be availed from the proposed DL with SCIM. Le and Hoang (2023) presented Metadata-hiding secure Advanced Privacy-preserving Learn (MAPLE), a novel metadata-hiding secure search engine with query functionality over data shared by many users with full policy limits. MAPLE provides metadata security at the time of query handling, securely securing metadata security by significantly reducing execution costs from state-of-the-art networks. Looking at tests conducted, the results indicated that MAPLE delivered higher functionalities and more reliable assurances with tangible performance, similar to its peers. Challenges facing cloud storage are realized and analyzed by Jalil et al. (2023). Based on the features, a basic categorization is realized: a reliable data model. The data integrity methods of this strategy were also analyzed and critiqued in light of important critical qualities that should be realized. The best result was achieved.

Gao et al. (2022) presented a dynamic utilization scheme of massively multiplayer online games (MMOGs) running over heterogeneous cloud platforms, having dynamic scaling down to even the hardware level of a virtual machine (VM) using Long Short-Term Memory (LSTM) to further improve energy consumption for quality of experience (QoE) requirements. The strategy compensated for the cost far from keypad players, many physical machines (PMs), and virtual machines (VMs), tight QoE requirements, and migrating costs while maintaining up to 44.8% of the cost-saving over standard oversupply while preserving gamers' QoE against task diversity. In such a multi-cloud environment, (Anwarbasha et al. 2021) have presented a "Dynamic Level Based Integrity Checking Protocol (DA-ICP)" for storing their data. The proposed solution was founded on the Provable Data Possession

(PDP) technique where a delegator, having delegated data with an unknown multi-cloud, could actually check or prove for sure that the original file in question still remains on the server without having to download the entire file. This will greatly reduce the cost, as it randomly picks up several blocks from the server side for statistical proof of existence. The suggested methodology brings with it better results than the other existing solutions since, with this, multi-cloud is being used in DA-ICP.

Mostafa et al., (2023) introduced a versatile multi-factor multi-layer verification system, amalgamating detection and entry control methods with automated identification selection. Its primary objective was to ensure a secure cloud service with reduced false alarms, impeding unauthorized access attempts. The proposed framework effectively detected potential malicious users and hackers, thwarting server and data breaches.

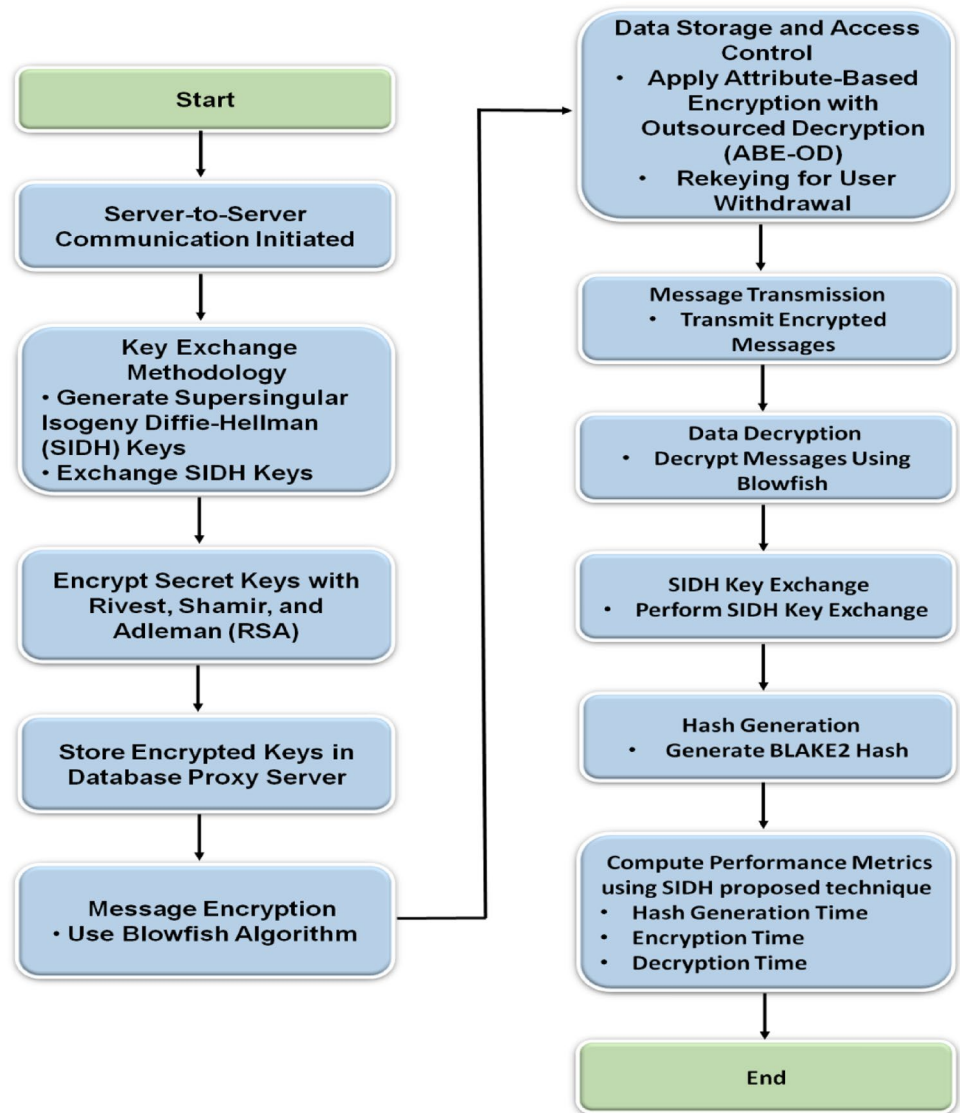
Megouache et al., (2020) presented a novel approach for ensuring authentication as well as data integrity within a dispersed and interoperable circumstance. Hashmi et al., (2024) described a unique steganographic technology called Product Cipher-Based Distributed Steganography (PCDS), which is intended to securely conceal data within a multi-cloud environment. The technique, which addresses the complexities of decentralized data hiding, uses intact cover material as standard over fragmenting as well as disguising data.

3 Methodology

The flowchart Fig. 1 for the multi-cloud security model details the sequential steps to ensure secure inter-server communication. The processes begin with server-to-server communication, followed through generating and exchanging keys with the SIDH technique. Secret keys are subsequently encrypted with RSA and stored within a database proxy server. Messages are encrypted utilizing the Blowfish and subjected to Attribute-Based Encryption with Outsourced Decryption (ABE-OD) for fine-grained access control and user rekeying. Encrypted messages are transmitted and decrypted with Blowfish. The SIDH key exchange is reiterated as part of the process. Finally, BLAKE2 hashes are generated, and performance metrics like hash generation, encryption, and decryption times are computed to evaluate the effectiveness of the proposed security model.

Data was gathered from safe multi-cloud simulation circumstances with a variety of service configurations and security settings. Performance indicators, security logs, as well as key management actions, were gathered from these

Fig. 1 Flowchart for this process



controlled simulations to evaluate the proposed security framework's performance across various situations.

3.1 Preliminaries

A bilinear map is a function which is linear in terms to every one of its inputs. In simpler terms, it integrates two distinct inputs to give an outcome which is linear in each argument. Bilinear maps are employed in a variety of cryptographic protocols, particularly in techniques employing pairing-based encryption, to enable operations across various algebraic structures. A Bilinear Map, consider H and HS to be multiplicative cyclic sets with prime sequence p . The features of a bilinear transfer $f : H * T \rightarrow HS$ are as follows. Bilinearity: There exists $f(h_b, h_a) = f(h, h)_{ba}$ for every $b, a \in YO$ and $h \in T$. Non-degeneracy: $f(h, h) \neq 1$, where V is the identity element of HS . Computable: f is efficient to

compute. Problems that are computationally hard this section discusses the two computationally difficult tasks listed below.

3.2 Integer factorization problem (IFP)

Assume p and q are ρ -bit primes, $M = o, r$, as well as $GenF$ is a polynomial-time algorithm that accepts ρ and returns (M, o, r) . The scaling condition for $GenF$ asserts that determining o and r from M is impossible to compute (hard) unless with low probability in ρ . The factoring advantage for a probabilistic polynomial-time (PPT) algorithm A is defined as:

$$Adv_{Gen_E, B}^{IFP}(\rho) = \Pr[(M, o, r) \leftarrow Gen_E(1^\rho) : A(M) = \{o, r\}] \quad (1)$$

The scaling requirement for $GenF$ says that $Adv_{GenE,B}^{IFP}(\rho)$ is insignificant in ρ for each PPT A . The expectation that $(t_{IFP}, \epsilon_{IFP}) - IFP$ obtains if $Adv_{GenE,B}^{IFP}(\rho) < \epsilon_{IFP}(\rho)$ for any small amount of $\epsilon_{IFP}(\rho)$.

3.3 Access structure

Consider the following parties: $\{O_1, O_2, \dots, O_n\}$. If $\mathbb{A} \subseteq 2^{\{O_1, O_2, \dots, O_n\}}$ and $\forall A, D$ satisfy if $A \in B$ and $A \subseteq D$, therefore $D \in B$, so we define B as monotonous. An access control hierarchy is an assortment of non-empty subgroups of $\{O_1, O_2, \dots, O_m\}$, defined as $B \subseteq 2^{\{O_1, O_2, \dots, O_n\}} \setminus \{\emptyset\}$. Similarly, the illegal set does not exist in $\mathbb{A}$. This article assumes the various participants' roles are based on their characteristic set. The access control hierarchy $\mathbb{A}$ consists of non-empty subsets of characteristics. This study focuses on the monotonous access structure for simplicity.

3.4 Attribute-Based encryption with outsourced decryption (ABE-OD)

Have T be a collection of characteristics, with B be a gateway object. The key input for the encryption method is (I_{enc}, I_{key}) . In ciphertext-policy ABE (CP-ABE), (I_{enc}, I_{key}) equals (B, T) , but in key-policy ABE (KP-ABE), (I_{enc}, I_{key}) equals (T, B) . The CP-ABE (KP-ABE) system with contracted decrypt comprises 5 methods:

- **Setup** ($1^\lambda, V$): The method accepts an attribute description V and a security factor λ . It produces a master key (mk) and a public key (pk). The method is executed by a reputable individual known as the private key generator (PKG).
- **Encryption** (pk, m, I_{enc}): The way to encrypt accepts a public key (pk), a message (m), and an access pattern (resp. feature set) I_{enc} . It generates the ciphertext (ct). The encryptor executes this method.
- **KeyGeneration** (mk, I_{key}): This method generates a key using a master key (mk) and an attribute set (I_{key}) for access. It generates the secret key (sk) and transform key (tk). PKG also performs this algorithm.
- **Transformation** (tk, ct): The method requires a transformation key (tk) for I_{key} with a ciphertext (ct) for I_{enc} . If $S \in A$, it produces. Ct_{out} contains partly decoded ciphertext. Otherwise, it generates a failed symbol ($\perp$). The method is executed by an agent using the transformation key tk from the person owning the private key sk .
- **Decryption** (sk, Ct_{out}): The decryption technique uses a private key (sk) for I_{key} along with part decrypted

ciphertext (ct_{out}) from I_{enc} . If $S \notin A$, it displays the error sign $\perp$. Otherwise, it generates a notification (m). The user performs this method using the private key sk .

ABE-OD, like ABE, relies on textual secrecy and it is indistinguishable from chosen-plaintext attacks. We have omitted a formal description because of space constraints.

4 RSA Process

1. Choose $p=5, q=17$
2. $n=p * q=5 * 17=85$
3. $\phi(n)=(p-1) * (q-1)=4 * 16=64$
4. $\exists e: (1 < e < \phi(n)) = > e \leftarrow 11, (e, n) \leftarrow \text{coprime}$
5. $\exists d: (d * e \bmod \phi(n)) = 1 = > d=35, \because [(35 * 11) \bmod 64 = 1]$
6. $\text{PublicKey} = (e, n) = (11, 85)$
7. $\text{PrivateKey} = (d, n) = (35, 85)$
8. choose $m=3$; $\text{encrypt}(m) = c = 3^{11} \bmod 85 \rightarrow c=7$
9. $\text{decrypt}(c) = m = 7^{35} \bmod 85 \rightarrow 37, 881, 869, 226, 566, 478 \% 85 \rightarrow m=3$

4.1 Elliptic curve cryptography: key generation

Elliptic Curve Cryptography is a kind of public-key cryptography based on the features of elliptic curves across finite fields. Elliptic Curve Cryptography utilizes the algebraic structure of these curves to generate safe keys and execute cryptographic operations. It is recognized for offering higher security with relatively smaller key dimensions and makes it more efficient than other cryptography approaches. Elliptic Curve Cryptography security is depending upon the complexity of solving certain mathematical issues involving these curves. Have E be a particular elliptic curve. It stretches over an infinite prime field (Fp). Let T be an intersection on the elliptic curve (Fp). T must possess a prime order n .

- 1 : $\text{KeyGen}(p, E, G, n)$
- 2 : $\text{PriKey}(dA) \leftarrow R | R = \text{random}([1, n-1]) \in Fp$
- 3 : $\text{LetBp}(G) = \text{BasePointofGenerator}G$
- 4 : $\text{PubKey}(HA) \leftarrow \text{PointMult}(\text{PriKey}(dA) * \text{Bp}(G))$
- 5 : $\text{return PubKey, PriKey}$
- 6 : end

Simple elliptic curve operations, such as point addition and doubling, are used to multiply.

4.2 Supersingular isogeny Diffie–Hellman (SIDH) key exchange

The conventional Diffie–Hellman protocol secures a shared secret between two entities through a public channel of communication that is susceptible to passive attacks. The SIDH protocol obtains the secret by calculating the j -invariant of two isomorphic super-singular curves with ellipses created by Bob and Alice, which are isogenous to a first super-singular curve E_0 . The following are the SIDH field parameters. Select an elliptic curve E that is supersingular over F_{p^2} .

$$o = (k_B)^{f_B} (k_A)^{f_A} e \pm 1 \quad (2)$$

In addition, e is a tiny cofactor, f_A and f_B are positive integers, and k_A and k_B are small prime numbers. Following that, $\#E = \left(k_A^{f_A} k_B^{f_B} e\right)^2$ is the cardinality of E . Let's define

$q_A = k_A^{f_A}$ and $q_B = k_B^{f_B}$ to help with notation simplicity. To produce the subgroups $E[q_A]$ and $E[q_B]$, select two pairs of independent elliptic curve points $\langle O_A, R_A \rangle = E[q_A]$, $\langle O_B, R_B \rangle = E[q_B]$. The public domain parameters include the prime p , curve E , and producing points O_A, R_A, O_B , and R_B . An isogeny $\phi : E \rightarrow E/\langle Q \rangle$ is computed as a composite of k -degree isogenies $\phi = \phi_{f-1}^{\circ} \cdot \phi_0$ for a point Q of order k^f . For $0 \leq j < f$, compute: $E_{i+1} = E_i / \langle k^{f-i-1} Q_j \rangle$, $\phi_j : E_j \rightarrow E_j + 1$, and $Q_j + 1 = \phi_j(Q_j)$. Let $E_0 = E$ and $Q_0 = Q$. Therefore, $E/\langle R \rangle = E_f$. Compute the curve $E_j + 1$ and isogeny ϕ_j in polynomial time using the point $k^{f-i-1} Q_j$. There are two primary phases to the SIDH key exchange mechanism. Both sides go forward as follows in the initial step, which is also referred to as the key generation phase:

- Alice chooses two random integers $n_A, m_A \in \mathbb{Z}_{q_A}$, and computes the isogeny $\phi_A : E \rightarrow E_A$, with kernel $\langle n_A O_A + m_A R_A \rangle = \langle Q_A \rangle$. Alice computes $\{\phi_A(O_B), \text{and } \phi_A(R_B)\}$ and transmits these to Bob, along with the calculated curve E_A .
- The isogeny $\phi_B : E \rightarrow E_B$ is computed by Bob using the kernel $\langle n_B O_B + m_B R_B \rangle = \langle Q_B \rangle$ after choosing two random integers, n_B , and m_B , that belong to $\mathbb{Z}_{q_B}$. Calculates $\{\phi_B(O_A), \text{and } \phi_B(R_A)\}$ and transmits these to Alice, along with the calculated curve E_B .

Bob and Alice calculate a shared secret as follows during the protocol's second phase:

- After Alice gets from Bob the values $\{E_B, \phi_B(O_A), \phi_B(R_A)\}$, she computes the

isogeny $\phi_{AB} : E_B \rightarrow E_{AB}$, with the kernel $\langle n_A \phi_B(O_A) + m_A \phi_B(R_A) \rangle = \phi_B(Q_A)$. Alice successfully obtains the shared secret using E_{AB} 's i -invariant function.

- After Alice gives Bob $\{E_A, \phi_A(O_B), \phi_A(R_B)\}$, he computes the isogeny $\phi_{BA} : E_A \rightarrow E_{BA}$, which has the kernel $\langle n_B \phi_A(O_B) + m_B \phi_A(R_B) \rangle = \phi_A(Q_B)$. Bob finally obtains the shared secret using E_{BA} 's i -invariant.

4.3 The process of implementation

The proposed architecture model comprises seven entities: Proxy Server, Certificate Authority, Attribute Authority, Trusted Authority, Data User, Data Owner and Storage Servers. A proxy server is a centralized entity that acts as a mediator between data users and cloud entities like Trusted Authority (TA), Attribute Authority (AA), and Storage Servers (SS), holding information and performing tasks. Certificate Authorities (CA) are trusted entities that issue public key certificates to data users and Attribute Authorities, requiring approval for registration and generating unique IDs and public keys. Attributes define data user access policies and AA issue attributes to users, update or revoke them. AA generates public and secret keys, files are encrypted with these keys, allowing only legitimate users to access them. TA is a reliable entity in cloud systems, encrypting secret keys with RSA and generating keys for attribute authority. It also acts as a judgment entity, validating data integrity by generating BLAKE2 hashes and comparing them with old hashes. Users can register, obtain unique IDs, certificates, and privileged attributes, as well as access data according to their access rights, with multiple attributes assigned by the AA. Data Owners (DO) manage and access data for remote and global access, assigning attributes to files for specific users. SS maintains encrypted file fragment databases, and requiring SIDH key exchange between PS and SS for secure communication. They do not store private keys, preventing decryption.

4.4 Proposed modeling of systems

Stage 1: system setup.

When establishing the structure, researchers take into account 4 key operations:

$TAInit(1^\lambda) \rightarrow (valk_{TA}, sigk_{TA})$. The Verified Entity is designed to be a trustworthy entity. The internal Security Parameter (λ) is used during the trusted authority initialization procedure. TA denotes trusted Authority. A validation key ($valk_{TA}$) and a signature key ($sigk_{TA}$). $CAInit(1^\lambda) \rightarrow (PSK, UP, (valk_{CA}, sigk_{CA}))$. During certificate authority establishment, an internal λ is required. The output includes the primary secret key (PSK), the universal parameter (UP), a validation key $valk_{CA}$, and a signature

Table 1 Notation and meanings

Notations	Meaning	Notations	Meaning	Notations	Meaning
$\widetilde{csk}$	New Content Secret Key	$PriK_{aid}$	Attribute Authority's Private Key	UPK_{uid}	Universal Public key of User
Det_v	User Details	$PubK_{aid}$	Attribute Authority's Public Key	USK_{uid}	Universal Secret key of User
D_B	Comprising Authority Set	AVK_{waid}	Attribute Version Key	Det_{AA}	Attribute Authority Details
$sigk_{CA}$	CA Signature Key	APK_{waid}	Attribute's Public Key	$MKUK$	Modifying Key to Update secret key
$valk_{CA}$	CA Validation Key	$AS_{uid,aid}$	Attribute Set	$SK_{uid,aid}$	Secret Key

$keysigk_{CA}.InitServers(PS, |SS|) \rightarrow$ Initialize the Proxy Server (PS) and Storage Servers (SS), where $|SS| = \{SS_1, SS_2, \dots, SS_n\}$. $UserSignUp(UP, sigk_{CA}, Det_u) \rightarrow (uid, UPK_{uid}, USK_{uid})$. The user signup algorithm requires inputs such as the universal parameter UP , the certificate authority's $sigk_{CA}$, and the user's signup details (Det_u). Upon user confirmation, a unique user ID (uid) is generated and assigned to them. The system generates a universal public key (UPK_{uid}) and a universal secret key (USK_{uid}) for each user. $AASignUp(Det_{AA}) \rightarrow (aid)$. The AA signup algorithm requires details (Det_{AA}) as input. Following validation, it creates aid and allocates it to the AA. $AAInit(UP, aid) \rightarrow s(PriK_{aid}, PubK_{aid}, \{AVK_{x_{aid}}, APK_{x_{aid}}\})$ as shown in Table 1. The AA initialization method takes the universal parameter UP and the AA's identity aid as inputs and generates a private key $PriK_{aid}$ and a public key $PubK_{aid}$. Additionally, it produces a pair of attribute versions and public keys for each attribute x .

Stage 2: secret KeyGeneration.

$SecKeyGen(PriK_{aid}, UP, \{APK_{x_{aid}}\}, AS_{uid,aid}) \rightarrow SK_{uid,aid}$. In this step, the system receives inputs such as AA's private key $PriK_{aid}$, the universal parameter UP , the attribute's public keys array $\{APK_{x_{aid}}\}$, array of attributes $AS_{uid,aid}$, and the

user ID uid are supplied to the system to generate the secret key $SK_{uid,aid}$.

Stage 3: Ciphering.

$$Enc\left(D, AP, UP, \{APK_k\}_{k \in CA}, \{APK_{x_k}\}_{x_k \in S_{A_k}}\right) \rightarrow EncD.$$

This function converts data (D) to encrypted data ($EncD$) by combining the D , access policy AP for specified characteristics, universal parameter UP , and public key set $\{APK_k\}_{k \in CA}$. The D is encrypted using Blowfish with a secret key csk , resulting in $EncD$. The content secret key (csk) is encrypted using the RSA technique while maintaining access policy (AP).

Key exchange suggested method $SIDHKE(PS, |SS|) \rightarrow ProxyServerPS$ checks array of Storage servers $|SS|$ via key exchange. If it validates $|SS| = Valid$, then, $Connect(PS, |SS|)$. $FragmentData(EncD, TF) \rightarrow EncryptedData$ is separated into a total number of fragments (TF). Fragment size (FR_{size}) is determined as $S(EncD)/TF$, where $S(EncD)$ is the size of encrypted data. The set of pieces is determined as $|FR| = \{FR_i, FR_{i+1}, FR_n\}$ Where $FR_i = F_{FR_{size}}$. For each fragment, a BLAKE2 hash is created using $HGen(|FR|)$ as follows:

```

forall FR in |FR| do
    HFRi = BLAKE2Process(FRi)end
    StoreHash(|HFRi|) → All the hashes and fragment IDs are stored by TA.
forall HFR in |HFR|do
    TAsave(FRID, HFRi, FID, TimeStamp)
end
StoreFragment(|FR|) → All the fragments and fragment IDs are stored by a set of Storage
servers|SS|.
forall FR in |FR|do
    forall SS in |SS|do
        SSi → SaveFrag(FRi)
        PSsave(FRID, HFRi, FID, TimeStamp)
    end
end

```

Stage 4: Deciphering.

In this phase, the cloud servers produce a Deciphering Request ID (DRID) to start the decryption of the data: $DRIDGen(EncD, UPK_{uid}, \{SK_{uid,k}\}_{k \in CA}) \rightarrow DRID$. The decoding request id creation function takes encrypted data $EncD$, access policy AP , user's universal public key UPK_{uid} , and secret keys set $\{SK_{uid,k}\}_{k \in CA}$. Users with valid uid in AP can obtain $DRIP$ for $EncD$.

Key exchange suggested method $ECDHKm(PS, |SS|) \rightarrow PS$ checks array of storage servers $|SS|$ via key exchange. If it confirms that $|SS| = Valid$, then $Connect(PS, |SS|)$. $FetchData(|SS|)$ the PS retrieves encrypted data pieces from an arbitrary storage server SS .

$|FR| = PS_{fetch(SS, FR_1, \dots, SS_n, FR_n)}$ The fragments are combined into a single encrypted data block.

$EncD = \sum_{i=1}^n FR_i Dec(EncD, DRID, USK_{uid}) \rightarrow D$. To convert encrypted data ($EncD$) to plain data (D), users must give their universal secret key (USK_{uid}). It uses the RSA technique to decode the content secret key (csk). The resulting csk is used to decipher $EncD$ using the BLAKE2 technique to create data D .

Stage 5: Attribute Revocation.

At this time, AA should update amended keys, update unrevoked user keys, and re-encrypt encrypted data.

$$\begin{aligned} &ModKeyGen(PriK_{aid}, \tilde{R}_{aid}, AVK_{\tilde{x}_{aid}}) \\ &\rightarrow (AVK_{(\tilde{x}_{aid})}, MKUK_{(\tilde{x}_{aid})}, \widetilde{csk}_{(\tilde{x}_{aid})}) \end{aligned} \quad (3)$$

This function takes the authority private key $PriK_{aid}$, the attribute $\tilde{R}_{aid}$, and the preceding version key $AVK_{\tilde{x}_{aid}}$. The system creates a new attribute version key $AVK_{(\tilde{x}_{aid})}$, a key for user modification of the secret key $MKUK_{(\tilde{x}_{aid})}$, and a new secret key ($\widetilde{csk}_{(\tilde{x}_{aid})}$).

$$ASKUpdate(SK_{uid, aid}, MKUK_{\tilde{x}_{aid}}) \rightarrow SK_{uid, aid}' \quad (4)$$

This function takes the present secret key ($SK_{uid, aid}$), and its updated key to update the key ($MKUK_{\tilde{x}_{aid}}$) for all non-revoked users in the group. A new secret key ($SK_{uid, aid}'$) is produced to prevent revoked users from accessing the data.

$$EncUpdate(EncD, csk, \widetilde{csk}_{(\tilde{x}_{aid})}) \rightarrow EncD' \quad (5)$$

The proxy server uses RSA to decrypt old csk and csk to decode encrypted data $EncD$. The decrypted data is re-encrypted using $\widetilde{csk}_{(\tilde{x}_{aid})}$ to create new encrypted data $EncD'$. Then, $\widetilde{csk}_{(\tilde{x}_{aid})}$ is encrypted using RSA.

Stage 6: Integrity Check.

During this stage, TA checks the integrity of fragments that are suspected of being tempered.

$HashCheck(SS_{ID}, FID) \rightarrow (VR)$. The fragment ID FID and storage server ID SS_{ID} are entered by this function and should be verified for consistency. TA requests PS to retrieve a fragment from a certain SS with the input ID, using the formula: $FR = PS_{Fetch(SS_{ID}, FR_i)}$

When FR is retrieved, PS transfers it to TA . TA produces a new hash for the fragment as $NH_{FR} = BLAKE2Prcess(FR)$. TA retrieves the fragment's previous hash using the formula $OH_{FR} = TA_{FetchHash(FR_i)}$. TA compares new and old hashes using the formula: $IntRes = comp(NH_{FR}, OH_{FR})$. If $IntRes = true$, the fragment is deemed real on that storage server. Otherwise, it is considered corrupted.

5 Result

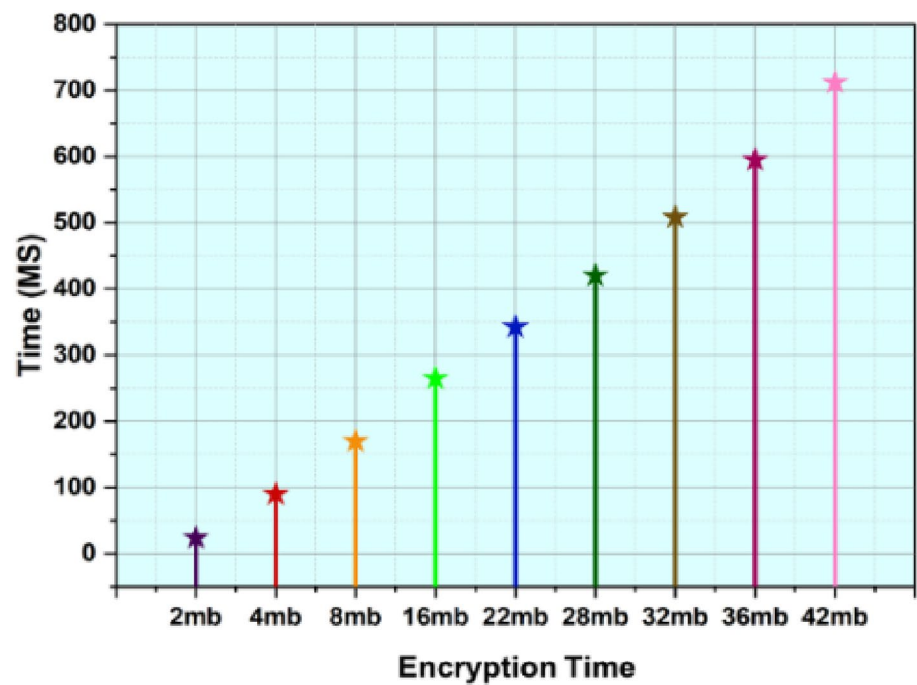
This study calculated the calculation times for BLAKE2 hash creation, data encryption, data decryption, and SIDH key exchange. Using an AMD Ryzen 5 3600 CPU running at 3.6 GHz and 16.0 GB RAM, we simulated a Windows-based machine. The code simulates SIDH techniques using the Bouncy Castle library (version v1.48). This research evaluated the efficiency of several SIDH curves. Running on Python (v3.11), every program was written in Python. The encryption, decryption, and hash generation results are averaged across 12 attempts with various file sizes.

Encryption time: Duration to convert plaintext to ciphertext using cryptographic techniques, varying from microseconds to milliseconds based on complexity and available processing resources. In Table 2 and Fig. 2 encryption time efficiency of the Blowfish algorithm is computed on various file sizes and the time taken is measured in milliseconds. Researchers measured encryption times of 2 mb (24), 4 mb (90), 8 mb (170), 16 mb (265), 22 mb (342), 28 mb (420), 32 mb (508), 36 mb (595), 42 mb (712).

Decryption time: Duration to revert ciphertext to plaintext using cryptographic methods, often similar to encryption time, contingent on algorithm complexity and system

Table 2 Encryption time

Encryption time	Time (ms)
2 mb	24
4 mb	90
8 mb	170
16 mb	265
22 mb	342
28 mb	420
32 mb	508
36 mb	595
42 mb	712

Fig. 2 Encryption time**Table 3** Decryption time

Decryption time	Time (ms)
2 mb	42
4 mb	164
8 mb	298
16 mb	417
22 mb	560
28 mb	682
32 mb	705
36 mb	907
42 mb	1024

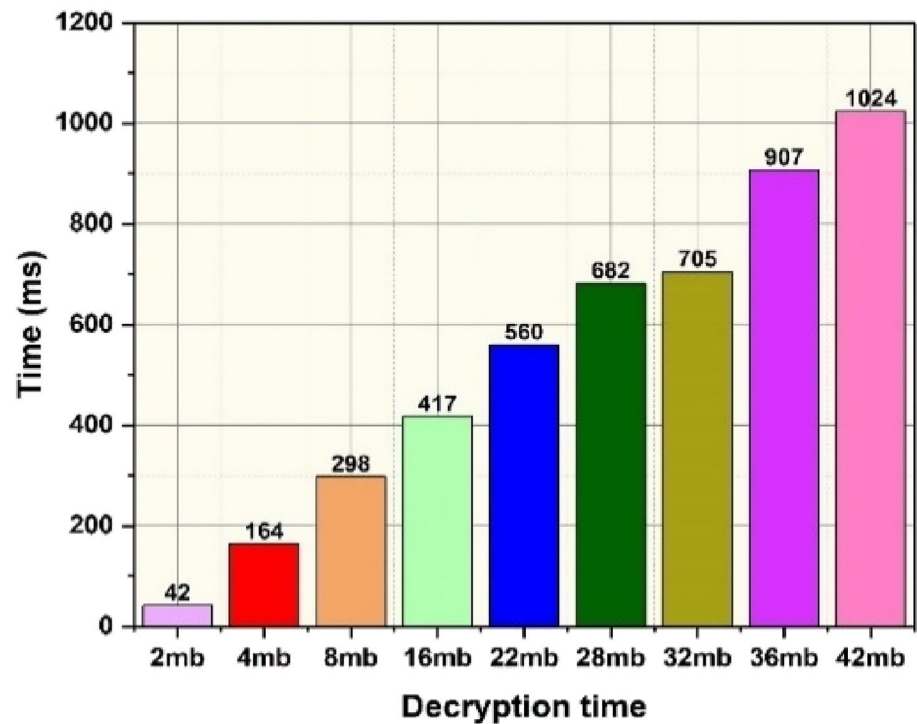
resources. In Table 3 and Fig. 3, the decryption time efficiency of the Blowfish algorithm is computed on various file sizes and the time taken is measured in milliseconds. Researchers measured decryption time of 2 mb (42), 4 mb (164), 8 mb (298), 16 mb (417), 22mb (560), 28 mb (682), 32 mb (705), 36 mb (907), 42 mb (1024).

The time required to calculate a cryptographic hash function on a given set of input data is referred to as the hash generation time. This time can range from milliseconds to microseconds, depending on the quantity of the input and the hashing algorithm's efficiency. In Table 4 and Fig. 4 hash generation time by the BLAKE2 algorithm is computed on various file sizes and the time taken is measured in milliseconds. Researchers measured hash generation time of 2 mb (45), 4 mb (167), 8 mb (301), 16 mb (421), 22 mb (563), 28 mb (685), 32 mb (705), 36 mb (911), 42 mb (1027).

This study compares the proposed SIDH algorithm with the existing Scalable and Secure Cloud Architecture (SSCA) (Irshad et al. 2023) technique for the response time and overall execution time. Table 5 displays the Response time and overall execution time. Figure 5 shows the response time (s) for the SSCA and proposed SIDH method. The achieved response time is 3.85 in SSCA and 2.01 in SIDH. Figure 6 shows the overall execution time for the SSCA and proposed SIDH method. The obtained overall execution time is 94.5 in SSCA and 55 in SIDH. The proposed SIDH outcomes attained better performance in response time and overall execution time.

6 Discussion

The article addressed the influence of the multi-cloud security architecture on authentication integrity, as well as the incorporation of sophisticated cryptographic methods such as bilinear maps, ABE-OD, and SIDH key exchanges. These approaches handle security issues such as key management as well as data accessing control, hence reducing risks such as unauthorized accessing and data leaks. The article highlighted the proposed architecture's practical advantages, capability to provide solid security ensures while managing complicated cloud interactions, and relevance to developing cloud security trends. Future research should concentrate on scalability and adaptability to changing threat scenarios.

Fig. 3 Decryption time**Table 4** Hash generation time

Encryption time	Time (ms)
2 mb	24
4 mb	90
8 mb	170
16 mb	265
22 mb	342
28 mb	420
32 mb	508
36 mb	595
42 mb	712

7 Conclusion

The provided frameworks for multi-cloud systems that require secure inter-server communication. Our security framework includes safeguarding data on storage devices and fine-grained mechanisms for controlling access, as well as a key transfer system for secure communication among servers. Using Public Key Infrastructure (PKI) forencrypted symmetric keys and inter-server key exchange ensures the reliability of the system. Conventional techniques like SIDH, Blowfish, RSA, and BLAKE2 provide a solid defense

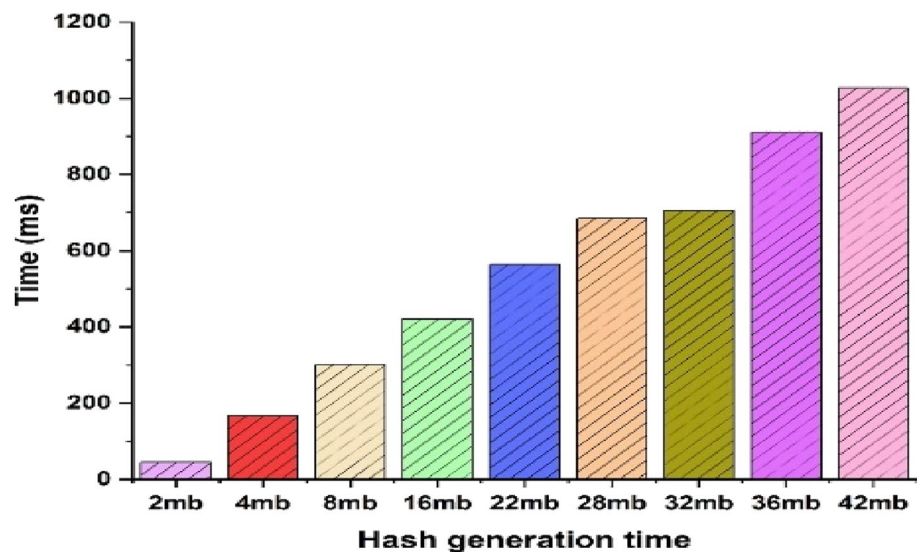
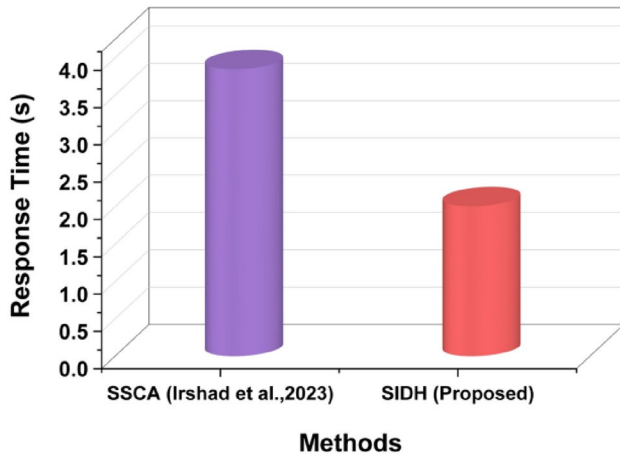
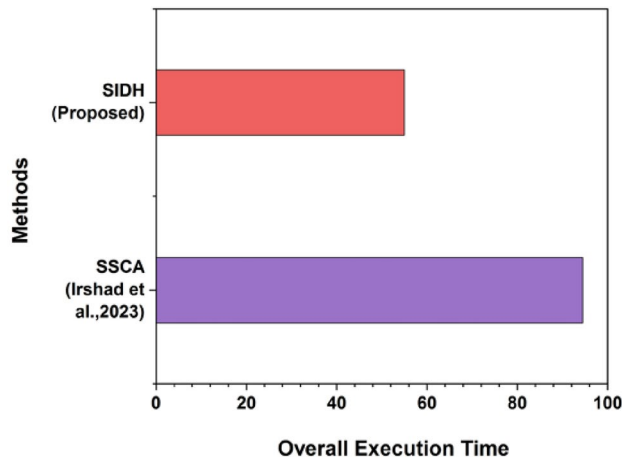
Fig. 4 Hash generation time

Table 5 Response time and overall execution time

Methods	Response time (s)	Overall execution time
SSCA (Irshad et al. 2023)	3.85	94.5
SIDH (proposed)	2.01	55

**Fig. 5** Response time**Fig. 6** Overall execution time

against assaults while maintaining excellent efficiency. The research we conducted and the findings show that the suggested method is sufficiently effective to be employed in actual time settings as a supplement to ABE-OD in multi-cloud environments. The outcomes of the parameters are hash generation time of 2 mb (45), 4 mb (167), 8 mb (301), 16 mb (421), 22 mb (563), 28 mb (685), 32 mb (705), 36 mb (911), 42 mb (1027), encryption times of 2 mb (24), 4 mb

(90), 8 mb (170), 16 mb (265), 22 mb (342), 28 mb (420), 32 mb (508), 36 mb (595), 42 mb (712), and decryption time of 2mb (42), 4 mb (164), 8 mb (298), 16mb (417), 22mb (560), 28mb (682), 32 mb (705), 36 mb (907), 42 mb (1024). Limitations include the possibility of message interception by attackers in ABE-OD-based multi-cloud systems, which pose a threat to data security and user privacy. The achieved response time is 2.01 in SIDH, overall execution time is 55 in SIDH. The proposed SIDH outcomes attained better performance. The proposed procedures, which have shown to be successful and efficient, aim to protect inter-server communication and maintain data integrity. To improve and expand the technique, future research must concentrate on verifying its applicability over a wider range of data and real-world settings to assure its ability to generalize and resilience. Incorporating newer approaches like deep learning (DL) or ensemble approaches could improve performance even more.

Acknowledgements There is no funding

Funding None.

Data availability Not applicable.

Code availability Not applicable.

Declarations

Conflict of interest The authors declare no competing interests.

Ethics approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

References

- AlRousan M, Intrigila B (2020) Multi-factor authentication for e-government services using a smartphone application and biometric identity verification. *J Comput Sci* 16(2):217–224
- Anwarbasha H, Sasi Kumar S, Dhanasekaran D (2021) An efficient and secure protocol for checking remote data integrity in multi-cloud environment. *Sci Rep* 11(1):13755. <https://doi.org/10.1038/s41598-021-93073-3>
- Gao Y, Wang L, Xie Z, Qi Z, Zhou J (2022) Energy-and quality of experience-aware dynamic resource allocation for massively multiplayer online games in heterogeneous cloud computing systems. *IEEE Trans Services Comput*. <https://doi.org/10.1109/TSC.2022.3190447>
- Harold Robinson Y, Golden Julie E (2019) MTPKM: multipart trust based public key management technique to reduce security vulnerability in mobile ad-hoc networks. *Wireless Personal Commun* 109(2):739–760. <https://doi.org/10.1007/s11277-019-06588-4>

- Hashmi SS, Khan Mohammad AA, Abdul AM, Atheeq C, Nizamuddin MK (2024) Enhancing data security in multi-cloud environments: a product cipher-based distributed steganography approach. *Int J Saf Secur Eng* 14(1):47–61. <https://doi.org/10.18280/ijssse.140105>
- Irshad RR, Hussain S, Hussain I, Nasir JA, Zeb A, Alalayah KM, Alatab AA, Yousif A, Alwayle IM (2023) Iot-enabled secure and scalable cloud architecture for multi-user systems: a hybrid post-quantum cryptographic and blockchain based approach towards a trustworthy cloud computing. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2023.3318755>
- Jalil BA, Hasan TM, Mahmood GS, Abed HN (2023) A survey on data integrity verification schemes in the cloud computing environment. In: *AIP conference proceedings* (Vol. 2475, No. 1). AIP Publishing. <https://doi.org/10.1063/5.0103392>
- Kang M, Lemieux V (2021) A decentralized identity-based blockchain solution for privacy-preserving licensing of individual-controlled data to prevent unauthorized secondary data usage. *Ledger*. <https://doi.org/10.5195/ledger.2021.239>
- Le T, Hoang T (2023) MAPLE: a metadata-hiding policy-controllable encrypted search platform with minimal trust. *Proc Privacy Enhanc Technol* 2023(4):184–203. <https://doi.org/10.56553/popets-2023-0105>
- Megouache L, Zitouni A, Djoudi M (2020) Ensuring user authentication and data integrity in multi-cloud environment. *HCIS* 10:1–20. <https://doi.org/10.1186/s13673-020-00224-y>
- Mhaisen N, Malluhi QM (2020) Data consistency in multi-cloud storage systems with passive servers and non-communicating clients. *IEEE Access* 8:164977–164986. <https://doi.org/10.1109/ACCESS.2020.3022463>
- Mostafa AM, Ezz M, Elbashir MK, Alruily M, Hamouda E, Alsarhani M, Said W (2023) Strengthening cloud security: an innovative multi-factor multi-layer authentication framework for cloud user authentication. *Appl Sci* 13(19):10871. <https://doi.org/10.3390/app131910871>
- Narayanan U, Paul V, Joseph S (2022) A novel system architecture for secure authentication and data sharing in cloud enabled Big Data Environment. *J King Saud Univ-Comput Inf Sci* 34(6):3121–3135. <https://doi.org/10.1016/j.jksuci.2020.05.005>
- Rajkumar N, Gokul M, Kumar KS, Mohanasuganthi N (2022a) Deep learning based user identity management protocol for improving security in cloud communication. *Int J Intell Syst Appl Eng* 10(3):444–453
- Rajkumar V, Prakash M, Vennila V (2022b) Secure data sharing with confidentiality, integrity and access control in cloud environment. *Comput Syst Sci Eng* 40(2):779–793. <https://doi.org/10.32604/csse.2022.019622>
- Shufutinsky A (2020) Employing use of self for transparency, rigor, trustworthiness, and credibility in qualitative organizational research methods. *Organiz Dev Rev* 52(1):50–58
- Vijay GR, Reddy ARM (2021) An extended infrastructure security scheme for multi-cloud systems with verifiable inter-server communication protocol. *Int J Comput Aided Eng Technol* 14(2):168–188. <https://doi.org/10.1504/IJCAET.2021.113544>
- Yang P, Xiong N, Ren J (2020) Data security and privacy protection for cloud storage: a survey. *IEEE Access* 8:131723–131740. <https://doi.org/10.1109/ACCESS.2020.3009876>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.